5.3.1 목표 및 1학기→2학기 전환

손가락이 굽혀질 때 피부 주름이 생기게 하는 것이 목표입니다. 엔진 기본 GPU 스키닝은 정점을 옮기기만 하며 표면 신장률을 계산하지 않으므로 주름이 생기지 않습니다.

1학기에는 SVE(Scene View Extension)를 이용해 포스트프로세스 단계에 커스텀 패스를 삽입하는 방식을 검증하였습니다. 2학기에는 계획대로 **후크 지점을 정점 처리 단계로 전환**, UE5의 `UMeshDeformer`를 상속하여 엔진 스킨 캐시를 커스텀 컴퓨트 셰이더로 대체하였습니다.

[그림 1] 최종 결과. 손을 편 상태에서 마디마다 가로 주름이 보입니다.

5.3.2 6-Pass 컴퓨트 셰이더 파이프라인

`UMeshDeformer`를 상속하여 컴퓨트 셰이더 6개를 글로벌 셰이더로 등록합니다.

```
IMPLEMENT_GLOBAL_SHADER(FPianoHandSkinDeformerCS,
"/PianoHand/Private/CustomSkinningDeformer.usf", "DeformCS",
SF_Compute);
IMPLEMENT_GLOBAL_SHADER(FPianoHandTensionAccumulateCS,
"/PianoHand/Private/CustomSkinningTension.usf",  "AccumulateCS",
SF_Compute);
IMPLEMENT_GLOBAL_SHADER(FPianoHandTensionResolveCS,
"/PianoHand/Private/CustomSkinningTension.usf",  "ResolveCS",
SF_Compute);
IMPLEMENT_GLOBAL_SHADER(FPianoHandDetailDisplaceCS,
"/PianoHand/Private/CustomSkinningDetail.usf",   "DisplaceCS",
SF_Compute);
IMPLEMENT_GLOBAL_SHADER(FPianoHandNormalAccumulateCS,
"/PianoHand/Private/CustomSkinningDetail.usf",   "NormalAccumulateCS",
SF_Compute);
IMPLEMENT_GLOBAL_SHADER(FPianoHandNormalResolveCS,
"/PianoHand/Private/CustomSkinningDetail.usf",   "NormalResolveCS",
SF_Compute);
```

순서	패스	스레드 단위	역할
①	DeformCS	정점	본 행렬로 위치·탄젠트 계산 (LBS / DQS)
②	AccumulateCS	삼각형	변 신장률을 계산해 정점에 누적
③	ResolveCS	정점	누적값 평균 → 정점 컬러 + strain 버퍼
④	DisplaceCS	정점	부피 보존 + 관절 주름 변위
⑤	NormalAccumulateCS	삼각형	변위 후 면 노멀 재계산
⑥	NormalResolveCS	정점	노멀 평균 + 탄젠트 재직교화

결과는 GPU skin passthrough 정점 팩토리 버퍼에 직접 씁니다. 머티리얼·그림자·라이팅 경로는 그대로 사용합니다.

```
FRDGBufferRef PositionBuffer =
FSkeletalMeshDeformerHelpers::AllocateVertexFactoryPositionBuffer(
    GraphBuilder, ExternalAccessQueue, MeshObject, LodIndex,
TEXT("PianoHandSkinDeformerPosition"));
// ... 6개 패스 디스패치 ...
FSkeletalMeshDeformerHelpers::UpdateVertexFactoryBufferOverrides(
    GraphBuilder, MeshObject, LodIndex, bInvalidatePreviousPosition);
```

> **Nanite를 쓰지 않은 이유:** Nanite 메시는 passthrough 정점 팩토리를 우회합니다. 정점 밀도는 에셋 단계 PN 테셀레이션으로 올렸습니다. ⑤⑥ 노멀 재계산 패스가 빠지면 정점이 움직여도 주름이 화면에 전혀 보이지 않습니다 — 초기 구현에서 실제로 겪은 문제입니다.

5.3.3 텐션(Strain) 계산

기준 자세의 변 길이와 스키닝 후 변 길이를 비교합니다.

```
// CustomSkinningTension.usf — AccumulateCS
void AccumulateEdge(uint A, uint B, float3 RestA, float3 RestB, float3 SkinA,
float3 SkinB)
{
    const float RestLen = length(RestB - RestA);
    const float SkinLen = length(SkinB - SkinA);
    if (RestLen < 1e-5f) return;                // 중복 정점으로 생긴 0길이 변은 제
외

    const float Strain = clamp(SkinLen / RestLen - 1.0f, -8.0f, 8.0f);
    const int   Packed = (int)round(Strain * TENSION_FIXED_SCALE);

    InterlockedAdd(StrainAccum[A], Packed);     // 여러 삼각형이 같은 정점에 동시에
쓴다
    InterlockedAdd(StrainAccum[B], Packed);
    InterlockedAdd(StrainCount[A], 1u);
    InterlockedAdd(StrainCount[B], 1u);
}
```

- **양수**: 늘어남 (손등·손가락 바깥)
- **음수**: 눌림 (손바닥·마디 안쪽)
- float atomic add는 GPU에서 사용 불가. `TENSION_FIXED_SCALE`(65536)을 곱해 int로 변환 후 atomic 누적. strain을 ±8로 clamp하면 정점에 변이 4,000개 붙어도 int32가 넘치지 않습니다.

`ResolveCS`에서 평균을 내고 정점 컬러로 출력합니다.



[그림 2] 텐션 디버깅. 왼쪽은 기준 자세로 변형이 없습니다. 오른쪽은 주먹을 쥔 상태입니다.
빨강이 늘어난 곳(손등·손가락 바깥), 파랑이 눌린 곳(손바닥·마디 안쪽)입니다.

5.3.4 관절 위치 판별 — 스킨 웨이트 기반

주름은 관절에만 생겨야 합니다. 별도 마스크를 쓰면 메시가 바뀔 때마다 다시 만들어야 합니다. 관절 위의 정점은 정의상 두 본이 섞이는 정점이므로, 스킨 웨이트에서 직접 뽑았습니다.

```
// CustomSkinningDetail.usf — DisplaceCS
// 강체 부분에서 0, 관절의 50:50 블렌드 선에서 정확히 1
const float JointBlend = saturate(4.0f * LargestWeight * SecondWeight);
const float JointBand  = pow(max(JointBlend, 0.0f), max(CreaseSharpness, 1.0f));
```

별도 마스크 없이 어떤 스켈레탈 메시에도 적용됩니다.

부피 보존: 면 방향 신장률이 λ이면 비압축 가정에서 두께는 $1/\lambda^2$배가 됩니다. 눌린 쪽이 법선 방향으로 부풀어 올라 LBS의 사탕포장지 변형을 방지합니다.

```
const float Lambda     = max(1.0f + Strain, 0.2f);
const float VolumeTerm = VolumeBulge * clamp(1.0f / (Lambda * Lambda) - 1.0f,
-2.0f, 2.0f);
```

5.3.5 이력(History) — 폈다 놓은 직후 남는 주름

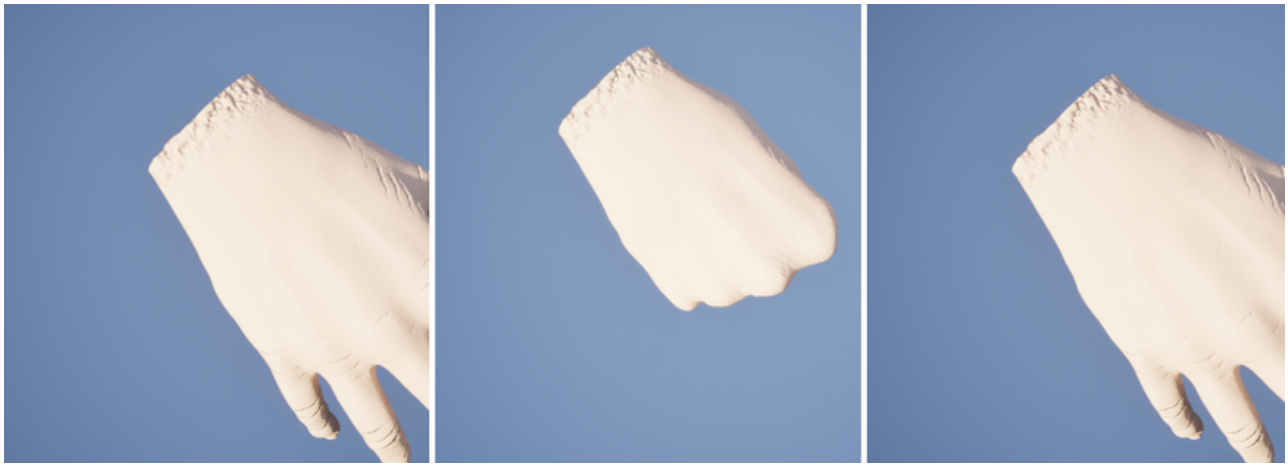
당겨졌던 피부는 바로 원래대로 돌아가지 않습니다. 잠시 주름이 남습니다. strain 하나로는 표현할 수 없습니다 — 쭉 펴고 있던 손가락과 방금 편 손가락의 strain이 같기 때문입니다. **직전 프레임 상태를 기억해야 합니다.**

```
// CustomSkinningTension.usf — ResolveCS
// RelaxState는 프레임 사이에 유지되는 버퍼 (.x = 직전 strain, .y = 남은 여유분)
const float2 PrevState   = (bResetRelax != 0u) ? float2(Strain, 0.0f) :
RelaxState[V];
const float  PrevStretch = max(PrevState.x, 0.0f);
const float  NowStretch  = max(Strain, 0.0f);

float Slack = PrevState.y;
Slack += max(PrevStretch - NowStretch, 0.0f) * RelaxGain;   // 신장이 풀리면 여유분
이 생기고
Slack -= max(NowStretch - PrevStretch, 0.0f) * RelaxGain;   // 다시 당기면 곧바로
사라진다
Slack  = saturate(Slack) * exp(-DeltaTime / max(RelaxTau, 1e-3f));

RelaxState[V] = float2(Strain, Slack);
ColorBufferUAV[V] = float4(saturate(T), saturate(-T), 0.5f + 0.5f * clamp(T, -1,
1), Slack);
```

버퍼를 프레임 사이에 유지하는 것이 핵심입니다. RDG의 `QueueBufferExtraction`과 `RegisterExternalBuffer`로 왕복시켰습니다. `Slack`은 정점 컬러 **알파 채널**로 나가 머티리얼이 주름을 되살리는 데 사용합니다.



[그림 3] 굴곡 사이클. 편 상태 / 주먹 / 막 편 직후입니다.
세 번째의 주름이 첫 번째보다 깊습니다. 이 차이가 이력(Slack) 버퍼의 효과입니다.

5.3.6 손 모델 교체 (MetaHuman)

기존 VR 템플릿 손은 관절이 단순한 2본 블렌드라 관절 밴드가 해부학과 어긋났습니다. MetaHumanCharacter 플러그인의 템플릿 바디에서 오른손만 잘라 교체했습니다.

단계	삼각형	정점
원본 바디 `SKM_Body`	60,816	30,455
오른손 추출	9,624	4,834
PN 테셀레이션 (level 3)	154,624	77,314



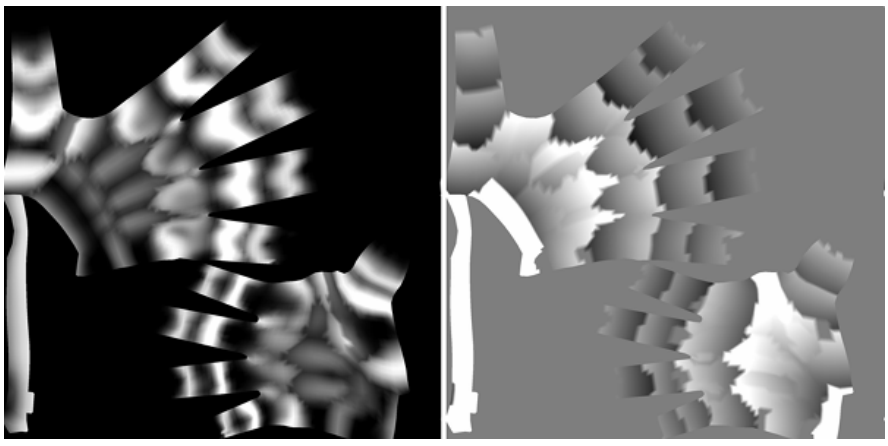
[그림 4] 추출한 오른손. 손목 절단면은 삼각형 팬으로 막았습니다.

두 가지 처리가 필요했습니다.

- **보조 본 정리**: MetaHuman은 정점 하나에 평균 9.6개 본을 묶습니다. 대부분 한 마디 안쪽의 보조 본이라 상위 두 웨이트가 같은 마디에 앉습니다. 평평한 손가락에서도 $4 \cdot w_0 \cdot w_1$이 1로 읽히는 문제가 발생합니다. 보조 본을 주 관절로 합쳐 평균 3.5개로 줄였습니다.
- **UV 재패킹**: 손 UV가 바디 아틀라스의 사선 조각이라 텍스처의 10%만 덮었습니다. 섬 회전을 포함해 재패킹해 47%로 올렸습니다.

관절별로 주름이 몰리는 정도는 리그에서 미리 계산해 텍스처로 구웠습니다.

채널	내용
R	가중치 적용 관절 밴드
G	관절 피벗 기준 뼈 축 방향 부호 거리 (±3 cm)
B	원본 관절 밴드
A	손등 / 손바닥 판별값



[그림 5] 구운 맵의 R 채널(좌)과 G 채널(우). 밴드가 각 마디에 앉아 있습니다. 셰이더는 G에 `sin()`을 적용해 손가락을 가로지르는 주름선을 만듭니다.

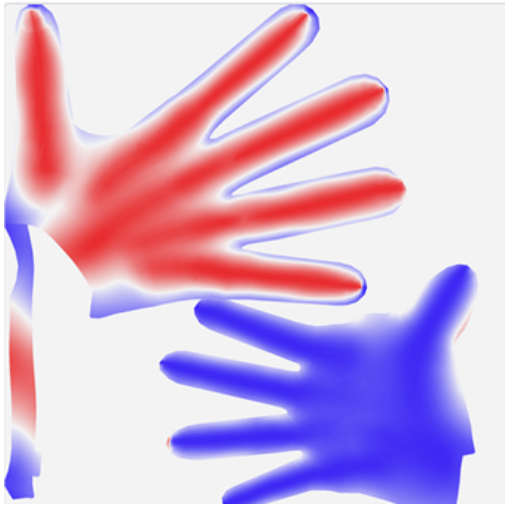
5.3.7 손등과 손바닥은 반대로 동작한다

관절 밴드는 스킨 웨이트에서 나오므로 관절을 빙 둘러 깔립니다. 실제 피부는 두 면이 반대로 움직입니다.

	손 펼 때	**주먹 쥘 때**
손바닥	주름 적음	주름 깊어짐
손등 (마디 위)	여유 피부가 뭉쳐 **주름 생김**	팽팽히 당겨져 **주름 사라짐**

베이크 단계에서 정점마다 "굽힐 때 늘어나는 쪽인지"를 계산해 마스크 알파에 넣었습니다. 굽힘은 본 로컬 +Z 축 기준 음수 회전 θ이며, 피벗에서 `u = cross(flexAxis, boneDir)` 방향으로 h만큼 떨어진 점은 `−h·θ`만큼 변합니다. `h > 0`이면 늘어나므로 u가 손등 방향입니다. 부호는 기준 자세에 40° 굽힘을 직접 적용해 검증하였습니다.

검증 항목	**값**
상관계수 `corr(dorsality, strain)`	**+0.68**
dorsality > +0.5 평균 변형률	**+6.9 %** (늘어남)
dorsality < −0.5 평균 변형률	**−9.4 %** (눌림)



[그림 6] 구워낸 판별 채널. 빨강이 손등, 파랑이 손바닥입니다.

셰이더 게이트를 손등/손바닥 둘로 나눕니다.

```
// PianoHandSkin.ush
float PH_CreaseGate(
    float Dorsal, float Compress, float Stretch, float Relax,
    float PalmarBase, float PalmarCompress, float PalmarIron, float PalmarRelax,
    float DorsalBase, float DorsalIron, float DorsalRelax)
{
    float Palm = PalmarBase + Compress * PalmarCompress + Relax * PalmarRelax -
Stretch * PalmarIron;
    float Back = DorsalBase                             + Relax * DorsalRelax -
Stretch * DorsalIron;
    return saturate(lerp(Palm, Back, Dorsal));
}
```

손등 쪽 `Back`에는 **압축 항이 없습니다**. 기본 여유분 `DorsalBase`에서 시작해 신장이 그것을 깎아냅니다.

파라미터	의미	값
`PalmarBase`	편 손바닥에 남는 기본 주름	0.30
`PalmarCompress`	쥘 때 손바닥 주름이 깊어지는 속도	2.6
`PalmarIron`	펼 때 펴지는 속도	1.2
`DorsalBase`	편 상태에서 손등에 뭉치는 주름	0.40
`DorsalIron`	쥘 때 팽팽해지는 속도	5.0
`DorsalRelax`	막 편 직후 되살아나는 주름	4.0
`SideSharpness`	손등↔손바닥 전환 폭	2.0



[그림 7] 게이트 하나였을 때(위)와 둘로 나눈 뒤(아래). 왼쪽이 편 손, 오른쪽이 주먹입니다. 수정 전에는 편 손이 매끈하고 주먹에서 주름이 보입니다. 수정 후에는 반대가 됩니다.



[그림 8] 손바닥. 같은 베이크 데이터로 반대로 동작합니다. 쥘수록 주름이 깊어집니다.

5.3.8 한계 및 향후 계획

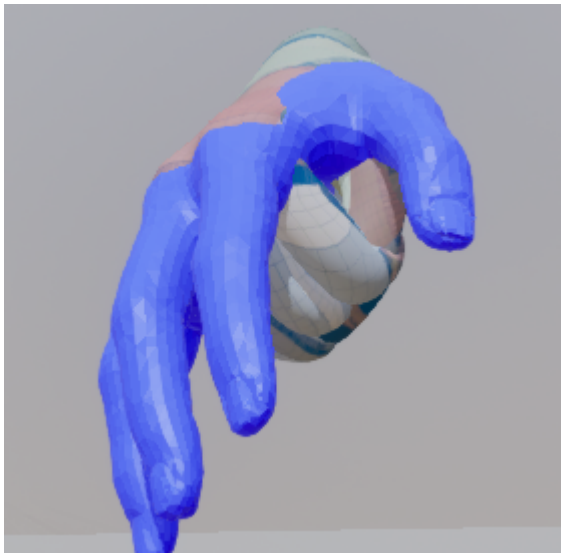
- **손바닥 접힘선**(생명선 등)은 관절 밴드로 나오지 않습니다. 별도 채널이나 수작업 맵이 필요합니다.
- **성능 측정 미완료**: 정점 77k에 컴퓨트 패스 6개의 프레임 비용을 측정해야 합니다.
- **이력 효과 감쇠 약 0.9초**: 정지 화면으로는 확인이 어렵습니다. 시퀀서 렌더로 검증할 예정입니다.
- **손목 절단면** 가장자리 노멀이 거칩니다. 커프를 씌우거나 경계를 평활할 계획입니다.
- 4.1·4.2 모듈의 실제 연주 모션 위에서 동작 확인 예정입니다.

5.4 Chaos Flesh PBD Skinning — 곽경민

5.4.1 목표 및 역할 분담

손가락이 굽혀질 때 살이 실제 물리처럼 눌리고 밀리게 하는 것이 목표입니다. LBS는 정점을 본 행렬로 옮기기만 합니다. 관절 안쪽 살이 서로 파고들고, 부피가 줄어듭니다. Chaos Flesh는 손 내부를 사면체로 채워 뼈에 묶고 물리로 시뮬레이션합니다. 눌린 살은 부피를 유지하려고 옆으로 밀려납니다.

이번 기간에는 통합 프로젝트에서 Flesh 에셋을 다시 만들고, 굴곡 애니메이션을 따라 Flesh가 변형되는 것까지 확인하였습니다.



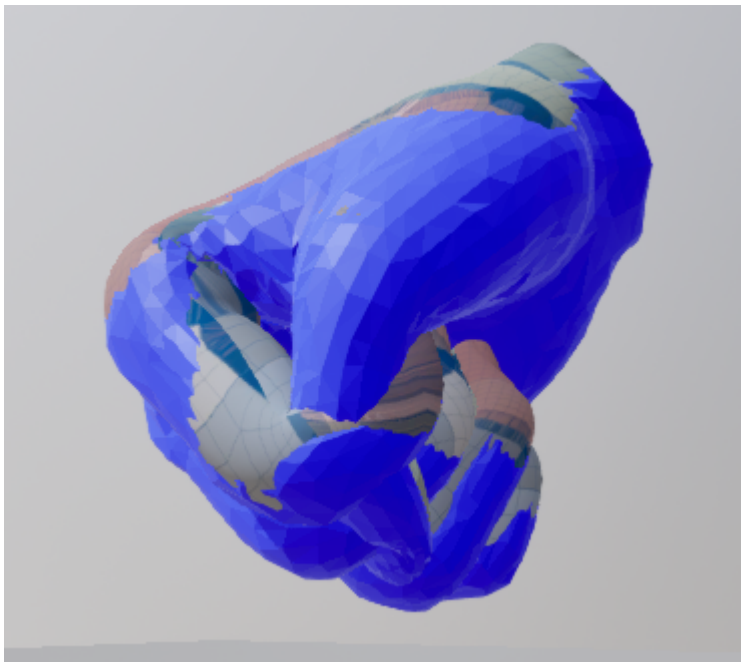


그림 1. 주먹을 쥐는 애니메이션을 Flesh(파랑)가 따라갑니다.

중간보고 이후 5.3의 커스텀 GPU 스키닝이 주름·핏줄까지 만들게 되었습니다. 두 모듈이 같은 일을 하지 않도록 역할을 나눴습니다.

	5.3 커스텀 GPU 스키닝	**5.4 Chaos Flesh**
변형 계산 근거	스킨 웨이트 (LBS/DQS) + 변 신장률	사면체 볼륨의 물리 시뮬레이션
표현 대상	주름·핏줄·미세 노멀	살 눌림·부피 보존
스케일	미시 디테일	거시 변형

5.3의 부피 보존 항은 $1/\lambda^2$라는 가정으로 계산한 **추정값**입니다. Flesh는 사면체끼리 서로 밀어내며 **실제로 계산**합니다. 최종 구조: **Flesh가 계산한 표면 위치 → 5.3 파이프라인 입력**으로 연결하여, 물리 변형된 표면 위에 주름·핏줄이 올라가는 구조를 목표로 합니다.

5.4.2 손 모델

5.3과 같은 메시를 씁니다. 나중에 결과를 합쳐야 하기 때문입니다. 5.3의 손 모델이 바뀔 때마다 따라갔습니다.

시기	**메시**	**정점**	**비고**
1학기	`fbx_Clean`	—	전신 모델, Bone Filter로 손·팔만 사면체화
2학기 초	`SKM_MannyXR_right`	3,239	UE 5.6 VR 템플릿 손
현재	`SKM_MH_Hand_R`	4,834	MetaHuman 바디에서 오른손만 추출

같은 메시가 필요한 이유는 5.4.6에 있습니다. Flesh는 **메시 이름**으로 애니메이션 뼈를 찾습니다. 시뮬레이션에는 PN 테셀레이션 전 원본 메시(`4,834 정점`)를 씁니다. 고밀도 메시(`77k 정점`)는 렌더링용입니다.

5.4.3 Dataflow 그래프

Flesh 에셋은 Dataflow 그래프로 만듭니다. 1학기에는 Geometry 생성과 바인딩을 그래프 두 개로 나눴습니다. 지금은 대상이 손 하나뿐이고 파라미터를 자주 바꿔야 해서 **하나로 통합**하였습니다.

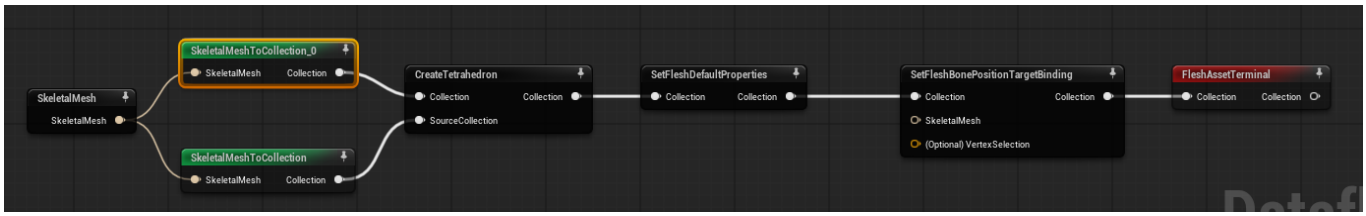


그림 2. SkeletalMesh → SkeletalMeshToCollection_0(Import Transform Only) + SkeletalMeshToCollection → CreateTetrahedron → SetFleshDefaultProperties → SetFleshBonePositionTargetBinding → FleshAssetTerminal

노드	역할	설정
`SkeletalMesh`	입력 모델	`SKM_MH_Hand_R`
`SkeletalMeshToCollection_0` (위)	뼈 계층만 변환	**Import Transform Only ✓**
`SkeletalMeshToCollection` (아래)	표면 형상 변환	기본값
`CreateTetrahedron`	표면 내부를 사면체로 채움	Method: **TetWild**
`SetFleshDefaultProperties`	물성 초기값 설정	아래 표
`SetFleshBonePositionTargetBinding`	사면체 정점 → 뼈 스프링	Stiffness 10,000 / Search Radius 1.0
`FleshAssetTerminal`	Flesh 에셋으로 저장	—

`CreateTetrahedron`의 두 입력은 역할이 다릅니다. 엔진 소스를 확인하였습니다.

```
// ChaosFleshCreateTetrahedronNode.cpp (UE 5.6, 발췌)
TUniquePtr<FFleshCollection> InCollectionVal(
    GetValue<DataType>(Context, &Collection).NewCopy<FFleshCollection>());
TUniquePtr<FFleshCollection> InSourceCollectionVal(
    GetValue<DataType>(Context, &SourceCollection).NewCopy<FFleshCollection>());
// SourceCollection의 형상으로 사면체를 만들어 InCollectionVal에 덧붙인다
SetValue<const DataType&>(Context, *InCollectionVal, &Collection);  // 출력 =
Collection 입력의 복사본 + 사면체
```

출력은 `Collection` 입력을 **그대로 복사한 것**에 사면체를 붙인 것입니다. 뼈 정보는 반드시 `Collection` 포트로 들어가야 합니다. 형상까지 든 컬렉션을 넣으면 원래 표면 메시가 사면체와 함께 중복으로 들어갑니다. 그래서 `Collection` 쪽은 **Import Transform Only**로 뼈만 넣습니다.

물성 파라미터:

파라미터	의미	값
`Density`	밀도	1.0
`Vertex Stiffness`	형태 유지 강성	1,000,000
`Vertex Damping`	진동 감쇠	0.0
`Vertex Incompressibility`	부피 보존 강도	0.6

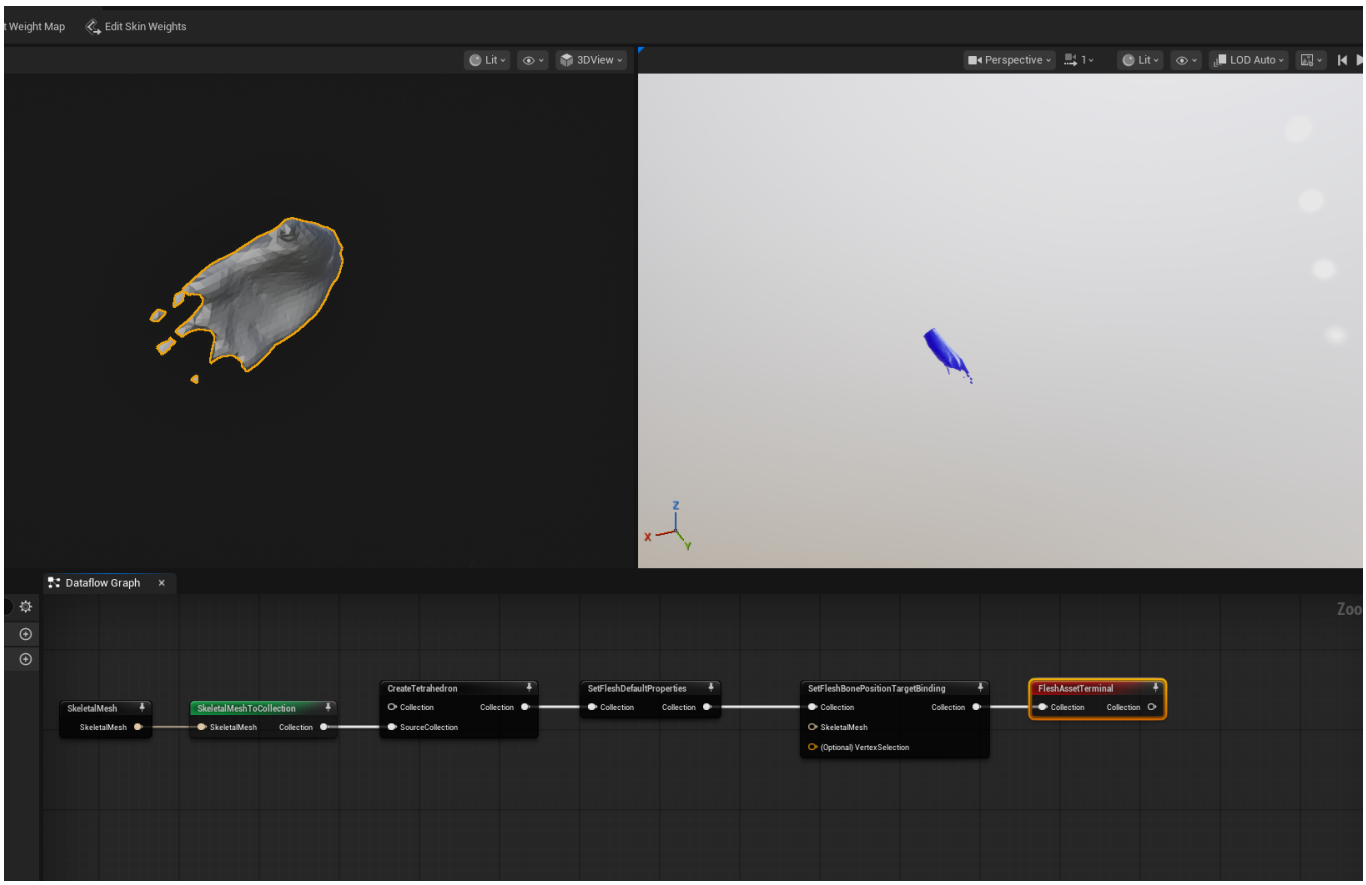
파라미터	의미	값
`Vertex Inflation`	내부 압력	0.5

5.4.4 사면체 생성 — IsoStuffing에서 TetWild로

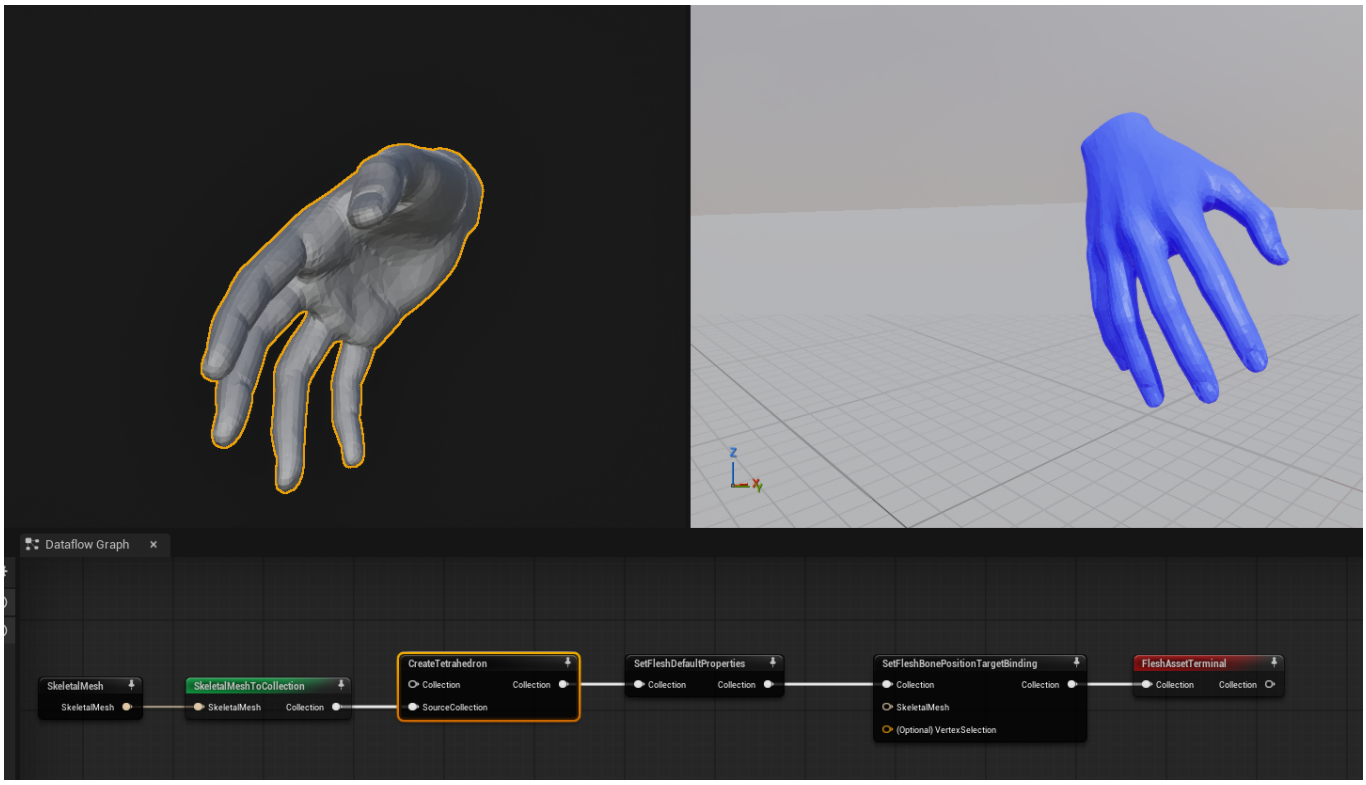
기본 방식은 IsoStuffing입니다. 바운딩 박스를 격자로 나누고 메시 안쪽에 있는 칸에 사면체를 채웁니다.

격자 해상도 문제: `Num Cells` 기본값 32에서 칸 크기는 약 0.6 cm(손 길이 약 19 cm 기준)입니다. 손가락 두께가 1.5~2 cm라 단면에 2~3칸밖에 안 들어갑니다. 48로 올려 칸을 약 0.4 cm로 줄였습니다. 3차원 격자라 칸 수는 $(48/32)^3 \approx 3.4$배가 됩니다.

MetaHuman 손에서 실패: 손 모델을 바꾸자 손바닥만 채워지고 손가락은 끝 조각만 남았습니다. IsoStuffing은 칸마다 "메시 안쪽인가"를 판정하므로 틈이 없는 닫힌 메시가 필요합니다. 이 메시는 전신 바디에서 잘라낸 것으로 표면에 틈이 남아 있어 안팎 판정이 실패한 것으로 분석됩니다.



틈이 있는 메시에도 안정적인 **TetWild**로 전환하였습니다.



5.4.5 뼈 바인딩

`SetFleshBonePositionTargetBinding`은 사면체 정점을 가까운 뼈에 스프링으로 묶습니다. **Position Target** 모드는 완전히 고정하지 않고 목표 위치로 당겨, 살이 약간 늦게 따라오는 효과가 납니다.

처음에는 손가락 끝이 아래로 처지고 뭉개졌습니다. 손끝은 마지막 뼈(`_03`)보다 바깥에 있어, 탐색 반경이 0이면 손끝 정점이 바인딩 범위 밖에 남습니다. Search Radius를 1.0으로 올려 해결하였습니다.

단, 이 조정은 뼈 정보가 빠진 상태(5.4.6)에서 한 것이므로 뼈 정보 복원 후 다시 검증해야 합니다.

5.4.6 애니메이션 추종

Flesh 에셋 에디터의 Preview Scene에서 굴곡 애니메이션(`A_MH_Hand_Cycle`, 4초 굴곡 사이클)을 재생하였습니다.

위치	항목	값
Asset Details → Animation	Skeletal Mesh	`SKM_MH_Hand_R`
Asset Details → Animation	Skeleton	`metahuman_base_skel`
Preview Scene	Animation Asset	`A_MH_Hand_Cycle`

처음에는 스켈레탈 메시만 주먹을 쥐고 Flesh는 기본 자세에 멈춰 있었습니다.

에디터 로그에 손의 뼈 22개 전부에 대해 같은 에러가 있었습니다.

```
ChaosFleshSetFleshBonePositionTargetBindingNodeLog: Error: Collection does not
contain bone[hand_r].
ChaosFleshSetFleshBonePositionTargetBindingNodeLog: Error: Collection does not
contain bone[index_01_r].
...
```

바인딩 노드는 컬렉션 안에서 **뼈 이름**으로 뼈를 찾습니다. 런타임에도 동일하게 동작합니다.

```
// ChaosDeformableTetrahedralComponent.cpp (UE 5.6, 발췌)
TSet<int32> Roots = TransformSource.GetTransformSource(
    Skeleton->GetName(), Skeleton->GetGuid().ToString(), SkeletalMesh-
>GetName());
if (!Roots.IsEmpty() && ...)
{
    const TArray<FTransform>& ComponentTransforms = SkeletalMeshComponent-
>GetComponentSpaceTransforms();
    AnimationTransforms[Adx] = ComponentTransforms[Cdx];   // 찾으면 애니메이션 뼈
사용
}
// 못 찾으면 AnimationTransforms는 기준 자세 그대로 → 경고 없이 멈춰 있는다
```

스켈레톤 이름 + GUID + 메시 이름으로 조회합니다. 5.4.2에서 5.3과 같은 메시를 써야 한다고 한 이유가 이것입니다. 그래프 통합 과정에서 `CreateTetrahedron`의 `Collection` 포트에 뼈 정보가 누락된 것이 원인이었습니다. `Import Transform Only` 갈래를 `Collection` 포트에 올바르게 연결하여 에러가 해소되고 Flesh가 애니메이션을 따라가는 것을 확인하였습니다.

5.4.7 한계 및 향후 계획

- **바인딩 파라미터 재조정 필요**: Stiffness와 Search Radius는 뼈가 빠진 상태에서 정한 값입니다.
- **부위별 바인딩 미구현**: 손끝·손목은 단단하게, 손등·손바닥은 유연하게 나눌 계획입니다. `VertexSelection` 입력으로 영역을 지정합니다.
- **물성 기본값에 가까움**: Damping이 0이라 초기 관통 해소 시 튀어 오르는 현상이 있습니다. Inflation은 얇은 손가락을 부풀릴 가능성이 있습니다. 추가 테스트 후 조정 예정입니다.
- **LBS 대비 비교 미완료**: 같은 프레임에서 관절부 형상을 나란히 비교할 예정입니다.
- **5.3 파이프라인 연동**: Flesh가 계산한 표면 위치를 5.3의 입력으로 넘기는 어댑터를 만들 예정입니다.
- **시뮬레이션 전용 메시**: TetWild는 생성이 느립니다. 틈을 메운 전용 메시를 따로 만들면 IsoStuffing으로 되돌릴 수 있습니다.

---

6. 주요 기술적 도전 및 해결

6.1 왼손 운지법 역전 문제 (01_Fingering)

현상: 왼손의 엄지(1번)가 가장 낮은 피치를, 새끼(5번)가 가장 높은 피치를 누르는 X자 꼬임 발생

원인: "낮은 피치 = 낮은 번호 손가락"이라는 오른손 중심 논리를 양손에 동일하게 적용. 왼손은 오른손의 거울상이므로 매핑 반전 필요.

해결: 왼손 연산 시 손가락 번호 조합 역순 배정 `(1,2,3)` → `(5,4,3)`, 교차 판정 로직 방향별 재구축.

6.2 IK 특이점 문제 (02_IK)

현상: 특정 자세에서 IK 계산값이 발산하여 불안정한 모션 생성

원인: 표준 Jacobian 역행렬 계산 시 특이점(Singularity) 근방에서 역행렬이 수치적으로 불안정합니다.

해결: $JJ^T + \lambda^2 I$ 형태의 **DLS** 역행렬을 적용하였습니다 ($\lambda = 0.05$). 또한 각도 변화량에 최대 스텝 제한을 두어 과도한 발산을 방지하였습니다. (이후 관절별 제한에서 9-DOF 전체 스텝 제한으로 변경 → 6.7절 참고)

6.3 GPU 셰이더 노멀 갱신 누락 (03_Skinning)

현상: 정점은 움직여도 주름이 화면에 전혀 보이지 않음

원인: 초기 구현에서 ⑤⑥ 노멀 재계산 패스 누락. 정점 위치가 변해도 노멀이 그대로면 라이팅 계산에 반영되지 않음.

해결: NormalAccumulateCS + NormalResolveCS 2개 패스 추가

6.4 손등·손바닥 주름 방향 반전 문제 (03_Skinning)

현상: 편 손에서 매끈하고 주먹에서만 주름이 보이는 비정상 동작. 실제 피부는 반대.

원인: 주름 게이트가 "눌리면 깊어진다" 단일 로직이었음. 손등과 손바닥의 구분 없이 같은 gate 적용.

해결: 베이크 단계에서 손등/손바닥 판별값(`dorsality`) 마스크를 구워 셰이더 게이트를 양면 분리.
`corr(dorsality, strain) = +0.68` 검증.

6.5 IsoStuffing 손가락 볼륨 생성 실패 (04_Flesh)

현상: MetaHuman 손 모델 적용 후 손가락 볼륨이 완전히 빔

원인: IsoStuffing의 안팎 판정이 표면 틈이 있는 메시에서 실패

해결: 비유사 메시에 안정적인 **TetWild** 방식으로 전환하여 5개 손가락 전체 볼륨 생성 확인

6.6 Chaos Flesh 애니메이션 추종 불가 (04_Flesh)

현상: 스켈레탈 메시만 움직이고 Flesh는 기본 자세에 멈춤

원인: 그래프 통합 과정에서 `CreateTetrahedron`의 `Collection` 포트에 뼈 정보가 누락됨. 바인딩 노드가 컬렉션 내 뼈 이름으로 탐색하는 구조.

해결: `Import Transform Only` 갈래를 `Collection` 포트에 올바르게 연결하여 뼈 계층 복원. 에러 해소 및 애니메이션 추종 확인.

6.7 IK 국소해 고착 문제 (02_IK)

현상: 양손 10개 손가락으로 확장한 뒤, 일부 손가락이 관절 한계각에 걸린 채 목표에 도달하지 못하였고, 이후 타임스텝에서도 이 상태에서 빠져나오지 못하였습니다.

원인: 반복별 디버그 출력으로 세 가지 원인을 확인하였습니다.

1. `std::clamp`만으로 관절 한계를 처리하여, 이미 한계에 닿은 관절에도 계속 움직임이 배정되고 클램프로 잘리는 무의미한 스텝이 반복되었습니다.
2. 최대 스텝 제한(0.1 rad)을 관절마다 따로 적용하여, 길이 1.0인 Joint0과 0.25인 Joint2가 매 반복 같은 각도만큼만 움직였습니다. 짧은 관절이 필요 이상으로 빨리 한계각에 닿았습니다.
3. 이전 타임스텝의 관절 각도에서 IK를 시작(웜스타트)하여, 한 번 빠진 나쁜 상태가 계속 이어졌습니다.

해결:

1. 관절 한계에 가까울수록 해당 관절의 참여를 줄이는 **Weighted DLS**를 적용하였습니다(5.2.3절). 하드 클램프는 마지막 안전장치로만 남겼습니다.
2. 9개 자유도 전체를 하나의 벡터로 보고 **전체 스텝 길이만** 제한하여, 관절 간 분배 비율을 유지하였습니다.
3. 각 타임스텝마다 관절 각도를 **0°로 초기화**한 뒤 새로 풀도록 하였습니다.

6.8 검은 건반 도달 실패 (02_IK)

현상: 일반 손가락이 검은 건반을 정확히 누르지 못하였고, 한 손가락만 건반을 누를 때도 옆으로 크게 뻗는 부자연스러운 자세가 나왔습니다.

원인: 검은 건반은 흰 건반 사이, 더 안쪽에 더 높게 있어 손가락이 옆과 앞으로 더 뻗어야 합니다. 그런데 MCP 외전 한계가 ±20°로 좁았습니다. 또한 손목을 누르는 건반들의 **평균 위치**에 두어, 각 손가락이 자기 `rootOffset`(최대 0.8)만큼 옆으로 더 뻗어야 했습니다.

해결:

1. MCP 외전 범위를 ±20°에서 **±35°**로 넓혔습니다(5.2.2절).
2. 손목을 **누르는 손가락의 뿌리가 건반 바로 위에 오는 위치**에 두도록 바꾸었습니다(5.2.6절).
3. 검은 건반을 누를 때는 손목이 앞쪽으로 차이의 30%만 이동하고 살짝 들리도록 하여, 나머지 거리는 손가락이 뻗어서 처리하게 하였습니다.

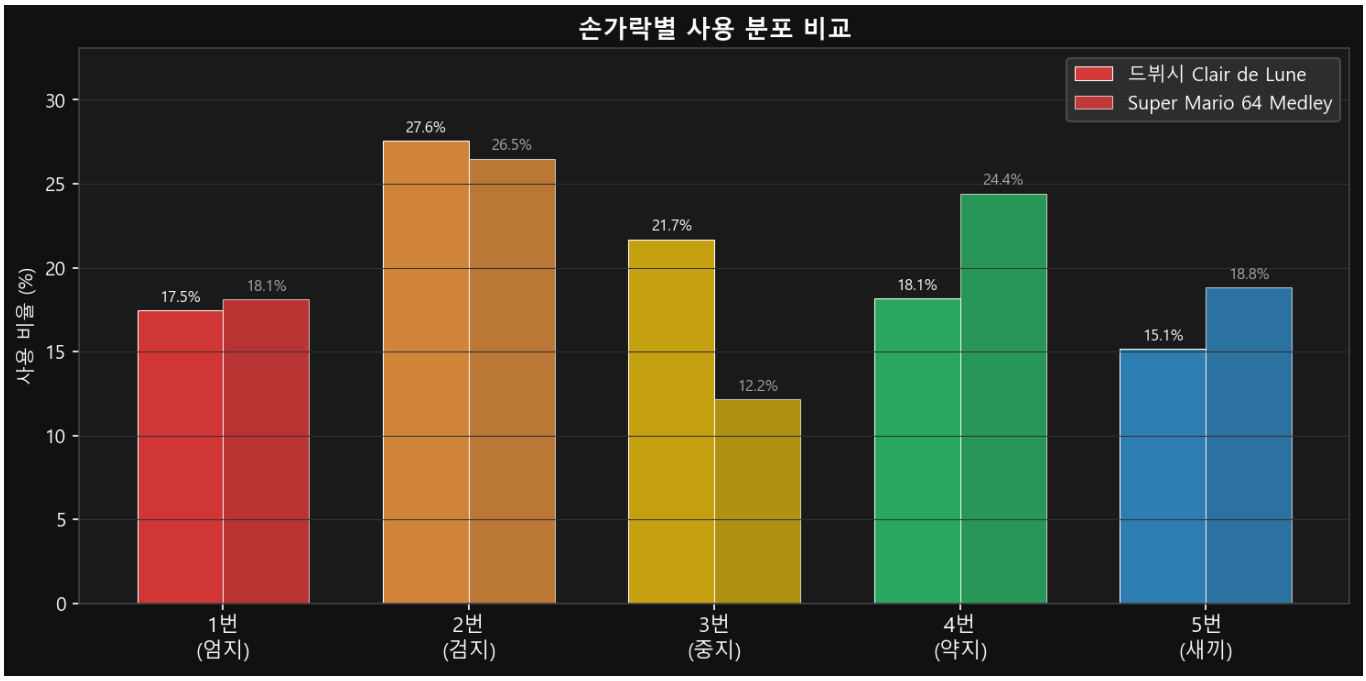
---

7. 결과물 분석

7.1 운지법 처리 결과 (12개 MIDI)

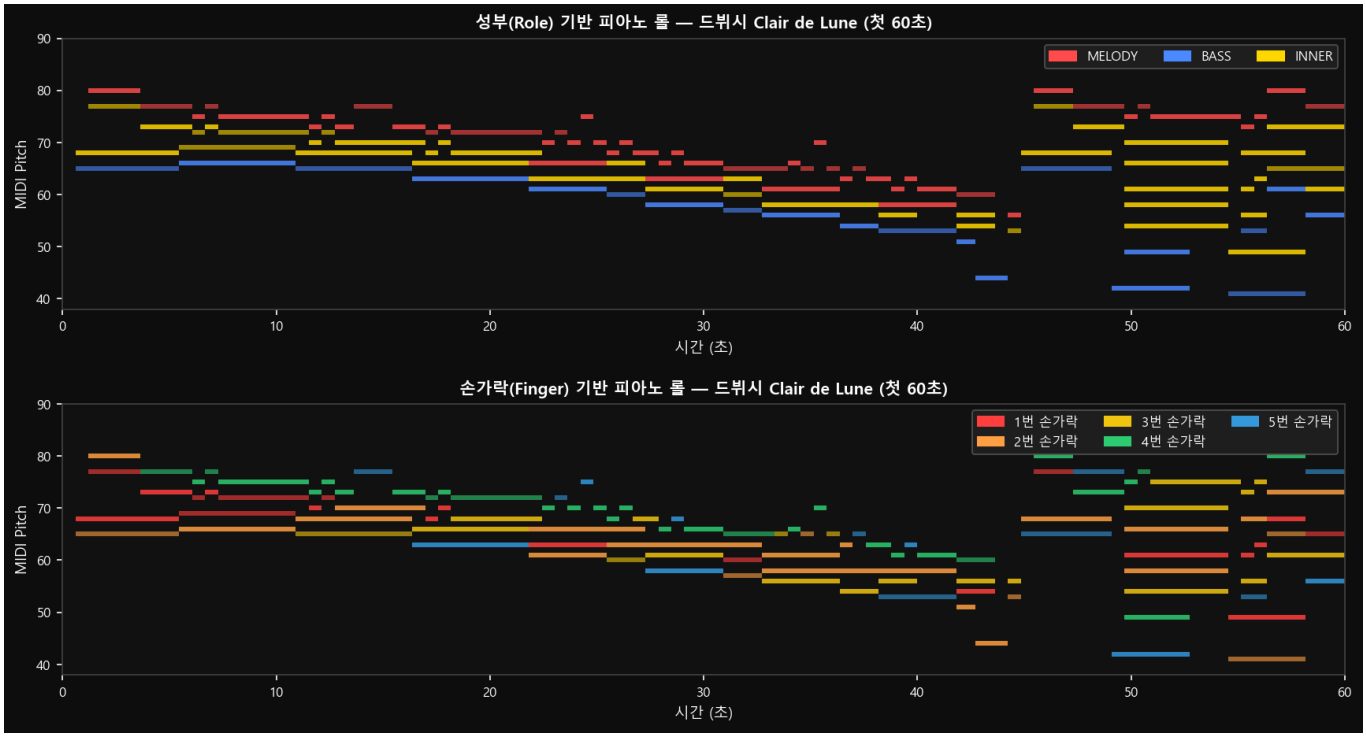
곡명	음표 수	곡 길이	왼손/오른손	흑건 비율
드뷔시 Clair de Lune	1,466	6.5분	763 / 703	73.9%
Super Mario 64 Medley	5,435	15.7분	996 / 4,439	17.8%
Queen Bohemian Rhapsody	—	—	—	—
Driftveil City BW	—	—	—	—
Wii Sports Theme	—	—	—	—
외 7곡	—	—	—	—

7.2 손가락 사용 분포



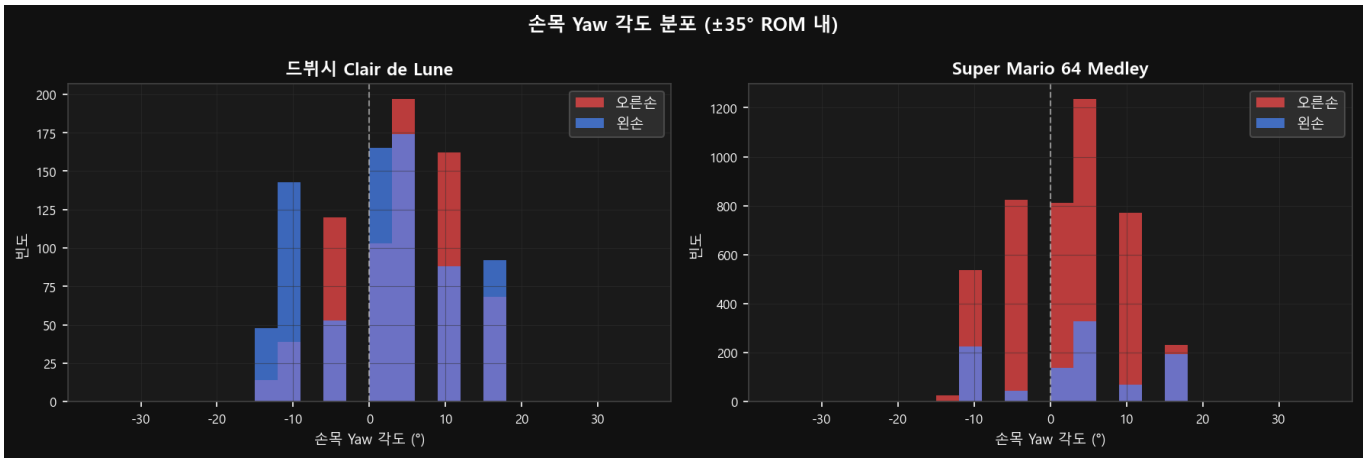
- 드뷔시: 흑건 비율 73.9%로 모든 손가락이 고르게 사용됨
- 마리오: 주선율이 오른손 집중 → 2번(26.5%), 4번(24.4%) 높음
- 약지(4번) 기피 패턴이 DP 비용(+6)으로 적절히 억제됨

7.3 피아노 롤 시각화



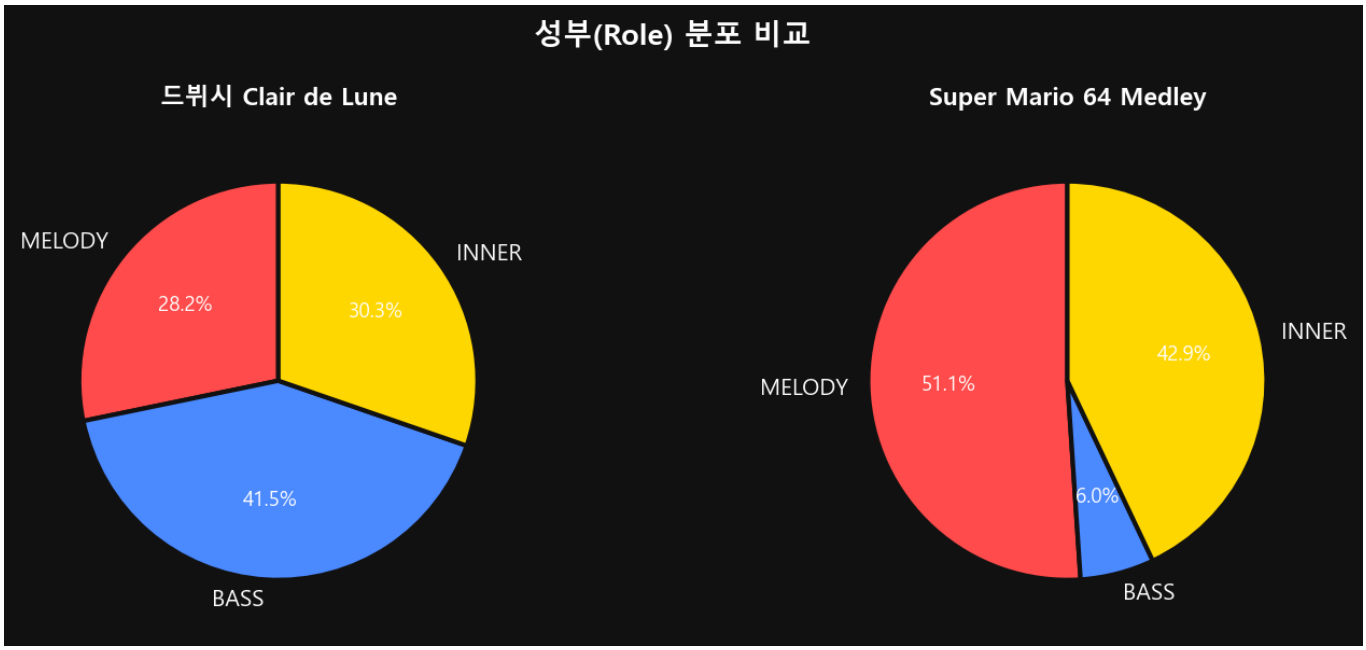
위: 성부(MELODY/BASS/INNER) 기반 채색 / 아래: 손가락 번호 기반 채색

7.4 손목 Yaw 각도 분포



손목 Yaw 각도가 ±35° ROM 내에서 정규 분포. 드뷔시의 분포가 더 넓은 것은 흑건 음형이 손목 회전을 더 많이 요구하기 때문입니다.

7.5 성부 역할 분포



7.6 GPU 스키닝 결과

- 편 손 상태에서 마디마다 가로 주름이 관찰됨 (손등 기본 주름)
- 주먹 시 손바닥 주름 깊어짐, 손등은 팽팽해지며 주름 사라짐
- 막 편 직후 약 0.9초간 이력 주름이 남아있다 서서히 소멸

7.7 Chaos Flesh 결과

- 주먹 쥐는 4초 굴곡 사이클 애니메이션에서 Flesh가 정상 추종 확인
- 사면체 5개 손가락 전체 채움 (TetWild, 방법 전환 후)
- Position Target 모드에서 살이 약간 늦게 따라오는 자연스러운 지연 효과 관찰

---

8. 현재 상태 및 향후 계획

8.1 완료 항목

- ☑ V5 Polyphonic DP 운지법 엔진
- ☑ Pygame 실시간 시뮬레이터
- ☑ shared_constants.py 공유 상수 분리 (W04)
- ☑ IK 인터페이스 파라미터 전항목 확정 (W03)
- ☑ 12곡 MIDI 운지법 일괄 계산
- ☑ UE5 DataTable CSV 출력 파이프라인
- ☑ 6-Pass GPU 스키닝 컴퓨트 셰이더 (DeformCS~NormalResolveCS)
- ☑ 텐션 맵 계산 (atomic int 고정소수점 누적)
- ☑ 이력 주름 효과 (RelaxState 프레임 간 버퍼 유지)
- ☑ 손등/손바닥 이중 주름 게이트
- ☑ MetaHuman 손 모델 추출 및 UV 재패킹
- ☑ Chaos Flesh Dataflow 그래프 (TetWild, 통합 단일)
- ☑ Chaos Flesh 애니메이션 추종 확인

8.2 진행 중

- ☐ 02_IK: UE5 Animation 시스템 연동 파이프라인 구축
- ☐ 04_Flesh: 바인딩 파라미터 재조정 (뼈 복원 후)
- ☐ 04_Flesh: 부위별 바인딩 (손끝 단단, 손등 유연)
- ☐ 03+04: Flesh → GPU 스키닝 어댑터 연결

8.3 다음 마일스톤

우선순위	모듈	항목
1	02_IK	UE5 AnimBP 연동, `start_ms` 기반 타임라인 트리거 → Control Rig 본 타겟 연결
2	03_Skinning	GPU 스키닝 성능 측정 (정점 77k, 6-pass 프레임 비용)
3	04_Flesh	바인딩 파라미터 재튜닝, 부위별 VertexSelection 설정
4	전체	Flesh 변형 표면 → GPU 스키닝 어댑터 구현
5	전체	엔드-투-엔드 데모: 운지법 → IK → 렌더링 통합 검증

8.4 알려진 한계

모듈	한계
01_Fingering	복잡한 대위법·현대 곡 대응력 추가 향상 여지
03_Skinning	손바닥 접힘선(생명선 등)은 관절 밴드로 불출력, 수작업 맵 필요
03_Skinning	이력 주름 감쇠 약 0.9s — 정지 화면 확인 불가, 시퀀서 렌더 검증 예정
03_Skinning	손목 절단면 노멀 거침 — 커프 씌우기 또는 경계 평활화 예정
04_Flesh	물성 기본값에 가까움 — Damping 0.0으로 초기 관통 해소 시 튀어 오르는 현상
04_Flesh	LBS 대비 비교 검증 미실시

---

9. 결론

본 중간 보고서는 피아노 연주 시뮬레이션 시스템의 2학기 W01~W04 진행 현황을 정리한 것입니다.

01_Fingering 모듈은 1학기에 완성된 V5 Polyphonic DP 엔진을 기반으로, 2학기에는 공유 상수 분리, IK 인터페이스 파라미터 확정, UE5 DataTable CSV 출력 파이프라인을 추가하여 운지법 파이프라인으로 완성하였습니다. 12개 MIDI 파일에 대한 일괄 처리 결과도 확보하였습니다.

02_IK 모듈은 DLS IK를 관절 한계를 고려한 Weighted DLS로 확장하고, 전체 스텝 제한과 타임스텝별 재초기화로 수렴 안정성을 확보하였습니다. MIDI 노트 이벤트와 실제 비율의 건반 모델을 연동하여, 양손 10개 손가락의 흰/검은 건반 타건과 손목 추적을 Vulkan 테스트베드에서 검증하였습니다. 향후 운지법 엔진 JSON 연동과 손목 회전 반영을 거쳐, 오프라인 계산 결과를 UE5로 전달할 예정입니다.

03_Skinning (GPU) 모듈은 1학기의 SVE 포스트프로세스 사전 검증을 발판 삼아, 2학기에는 UMeshDeformer 기반 6-Pass 컴퓨트 셰이더를 완성하였습니다. 텐션 계산, 부피 보존, 이력 주름, 손등/손바닥 이중 게이트까지 구현되어 시각적으로 사실적인 주름 렌더링을 달성하였습니다.

03_Skinning (Flesh) 모듈은 IsoStuffing에서 TetWild로 전환하여 MetaHuman 메시에서의 사면체 생성 문제를 해결하고, 애니메이션 추종 기능을 확인하였습니다. 바인딩 파라미터 재조정 및 부위별 바인딩 적용이 남은 과제입니다.

2학기 최종 발표까지 IK-UE5 연동 완성, Flesh-GPU 스키닝 어댑터 구현, 그리고 MIDI 입력부터 최종 렌더링까지의 엔드-투-엔드 데모 시연을 목표로 개발을 지속하겠습니다.

졸업프로젝트[3201] 2팀 | 정근녕, 한승현, 이수민, 곽경민

본 보고서는 2026년 9월 29일 기준으로 작성되었습니다.